План реализации системы офлайн-обновлений

Версия документа: 1.0.0
Дата создания: 2025.12.30
Проект: FSA-AstraInstall
Компания: ООО "НПА Вира-Реалтайм"
Текущая версия проекта: V3.7.208 (2025.12.25)
Статус: 📝 ПЛАН ГОТОВ К РЕАЛИЗАЦИИ

📋 Оглавление

1. Цель реализации
2. Текущее состояние
3. Проблема
4. Архитектура решения
5. Структура данных
6. Пошаговый план реализации
7. Детальные изменения кода
8. Интеграция с GUI
9. Примеры использования
10. Тестирование
11. Риски и митигация

🎯 Цель реализации

Создать систему офлайн-обновлений операционной системы, которая:

- ✅ Позволяет подготовить пакет обновлений на системе с интернетом
- ✅ Создает отдельные пакеты для Astra Linux 1.7 и 1.8
- ✅ Интегрируется с существующей структурой `AstraPack/`
- ✅ Автоматически определяет версию системы и выбирает нужный пакет
- ✅ Обеспечивает полное обновление системы без интернета
- ✅ Позволяет развертывать полностью готовую сборку (ОС + компоненты) офлайн

📊 Текущее состояние

Существующая функциональность обновления:

Компонент	Описание	Файл	Строки
`SystemUpdater`	Класс обновления системы	FSA-AstraInstall.py	36038-37900
`update_system()`	Основной метод обновления	FSA-AstraInstall.py	37060-37289

Компонент	Описание	Файл	Строки
`_check_network_repositories()`	Проверка сетевых репозиториев	FSA-AstraInstall.py	36999-37057
`check_system_update_needed()`	Проверка наличия обновлений	FSA-AstraInstall.py	36167-36220
`detect_astra_version()`	Определение версии системы	FSA-AstraInstall.py	39183-39252

Процесс обновления (текущий):

```
1. Проверка сетевых репозиториев (_check_network_repositories)
2. Проверка наличия обновлений (check_system_update_needed)
3. apt-get update (обновление списков пакетов)
4. apt-get dist-upgrade -y (обновление системы)
5. apt-get autoremove -y (очистка)
```

Структура AstraPack:

```
AstraPack/
├── Astra/              # Компоненты Astra.IDE
├── Wine/               # Wine пакеты
├── Cont/               # CONT-Designer
└── Winetricks/         # Winetricks и кэш
```

🔍 Проблема

Проблема 1: Невозможность обновления без интернета

Симптомы:

- Требуется интернет для `apt-get update` и `apt-get dist-upgrade`
- Системы без доступа к репозиториям не могут быть обновлены
- Изолированные контуры требуют ручного обновления пакетов

Текущее ограничение:

```
# FSA-AstraInstall.py, строка 37066
if not self._check_network_repositories():
    print("[ERROR] Нет сетевых репозиториев!")
    print("[ERROR] Обновление системы невозможно без сетевых
репозиториев")
    return False
```

Проблема 2: Разные пакеты для разных версий

Факторы:

- Astra Linux 1.7 (GLIBC 2.28, базируется на Debian Buster)
- Astra Linux 1.8 (GLIBC 2.36, базируется на Debian Bookworm)
- Пакеты обновлений различаются между версиями
- Необходимо подготавливать отдельные пакеты

Проблема 3: Отсутствие интеграции с AstraPack

Недостаток:

- AstraPack содержит компоненты приложений, но не системные обновления
- Нет единой точки хранения всех необходимых пакетов
- Невозможно создать полностью автономную сборку

🏗️ Архитектура решения

Обзор

Система офлайн-обновлений состоит из двух основных компонентов:

1. **OfflineUpdatePackager** — класс для подготовки пакетов обновлений
2. **Модифицированный SystemUpdater** — поддержка локальных репозиториев

Схема работы

```
┌──────────────────────────────────────────────────────────────┐
│              ПОДГОТОВКА ПАКЕТА (система с интернетом)        │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
                   ┌──────────────────────┐
                   │ Определение версии   │
                   │ системы (1.7 / 1.8)  │
                   └──────────────────────┘
                              │
                              ▼
                   ┌──────────────────────┐
                   │ apt list --upgradable│
                   │ (список обновлений)  │
                   └──────────────────────┘
                              │
                              ▼
                   ┌──────────────────────┐
                   │ apt-get download     │
                   │ (скачивание всех deb)│
                   └──────────────────────┘
                              │
                              ▼
```

```
                    ┌──────────────────────┐
                    │ Создание структуры   │
                    │ локального репозитория│
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │ dpkg-scanpackages    │
                    │ (генерация индексов) │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │ Упаковка в tar.gz    │
                    │ + metadata.json      │
                    └──────────────────────┘
                              │
                              ▼
          ┌──────────────────────────────────────┐
          │ AstraPack/SystemUpdates/             │
          │   offline-update-1-7-YYYYMMDD.tar.gz │
          │   offline-update-1-8-YYYYMMDD.tar.gz │
          └──────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────┐
│        ИСПОЛЬЗОВАНИЕ ПАКЕТА (система без интернета)          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │ Определение версии   │
                    │ системы (1.7 / 1.8)  │
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │ Поиск пакета в       │
                    │ AstraPack/SystemUpdates/│
                    └──────────────────────┘
                              │
                              ▼
                    ┌──────────────────────┐
                    │  Пакет найден?       │
                    └──────────────────────┘
                     │                    │
              ДА     │                    │ НЕТ
                     ▼                    ▼
          ┌──────────────────┐   ┌──────────────────┐
          │ Распаковка в /tmp/ │   │ Обычное обновление│
          └──────────────────┘   │ через интернет   │
                    │            └──────────────────┘
                    ▼
          ┌──────────────────────┐
          │ Создание локального  │
          │ репозитория в /opt/  │
          └──────────────────────┘
                    │
```

```
                    │
                    ▼
          ┌──────────────────┐
          │ Backup sources.list│
          └──────────────────┘
                    │
                    ▼
          ┌──────────────────┐
          │ Добавление локального│
          │ репозитория в sources│
          └──────────────────┘
                    │
                    ▼
          ┌──────────────────┐
          │ apt-get update    │
          └──────────────────┘
                    │
                    ▼
          ┌──────────────────┐
          │ apt-get dist-upgrade│
          └──────────────────┘
                    │
                    ▼
          ┌──────────────────┐
          │ apt-get autoremove │
          └──────────────────┘
                    │
                    ▼
          ┌──────────────────┐
          │ Restore sources.list│
          └──────────────────┘
                    │
                    ▼
          ┌──────────────────┐
          │ Cleanup temp files │
          └──────────────────┘
```

📁 Структура данных

Структура офлайн-пакета

```
offline-update-1-7-20251230.tar.gz
│
├── metadata.json                    # Метаданные пакета
│   {
│     "version": "1-7",              # Версия Astra Linux
│     "date": "2025-12-30",          # Дата создания
│     "packages_count": 1250,        # Количество пакетов
│     "total_size_mb": 2100,         # Размер в MB
│     "base_system": "Debian Buster",# Базовая система
```

```
│     "glibc_version": "2.28",       # Версия GLIBC
│     "created_by": "FSA-AstraInstall V3.7.208"
│   }
│
├── deb/                             # Все deb файлы
│   └── pool/
│       └── main/
│           ├── a/
│           │   ├── apt_1.8.2.3_amd64.deb
│           │   └── ...
│           ├── b/
│           │   ├── bash_5.0-4_amd64.deb
│           │   └── ...
│           └── ...
│
├── repo/                            # Структура APT репозитория
│   └── dists/
│       └── local-update/
│           └── main/
│               └── binary-amd64/
│                   ├── Packages       # Индекс пакетов (текст)
│                   ├── Packages.gz    # Индекс пакетов (сжатый)
│                   └── Release        # Метаданные репозитория
│
└── README.txt                       # Инструкция по использованию
```

Структура AstraPack после реализации

```
AstraPack/
├── SystemUpdates/                   # НОВАЯ ПАПКА
│   ├── offline-update-1-7-20251230.tar.gz
│   ├── offline-update-1-8-20251230.tar.gz
│   └── README.md                    # Инструкция
│
├── Astra/                           # Существующие компоненты
│   └── astra_packages.tar.gz
│
├── Wine/
│   └── wine_packages.tar.gz
│
├── Cont/
│   └── CountPack.tar.gz
│
└── Winetricks/
    └── winetricks_packages.tar.gz
```

📝 Пошаговый план реализации

Этап 1: Создание класса OfflineUpdatePackager ⏱️ 2-3 часа

Цель: Реализовать класс для подготовки офлайн-пакетов обновлений.

Файл: FSA-AstraInstall.py (добавить после класса SystemUpdater)

Задачи:

1. Создать класс `OfflineUpdatePackager`
2. Реализовать метод `prepare_offline_package()`
3. Реализовать метод `_get_upgradable_packages()`
4. Реализовать метод `_download_packages()`
5. Реализовать метод `_create_repository_structure()`
6. Реализовать метод `_generate_repository_indexes()`
7. Реализовать метод `_create_archive()`
8. Добавить обработку ошибок и логирование

Зависимости:

- `detect_astra_version()` — уже существует (строка 39183)
- Системные утилиты: `apt-get`, `dpkg-scanpackages`, `gzip`

Этап 2: Модификация SystemUpdater ⏱️ 2-3 часа

Цель: Добавить поддержку локальных репозиториев в существующий процесс обновления.

Файл: FSA-AstraInstall.py (модификация класса SystemUpdater)

Задачи:

1. Добавить метод `_find_offline_update_package()`
2. Добавить метод `_extract_offline_package()`
3. Добавить метод `_setup_local_repository()`
4. Добавить метод `_backup_sources_list()`
5. Добавить метод `_add_local_repository_to_sources()`
6. Добавить метод `_restore_sources_list()`
7. Добавить метод `_update_from_offline_package()`
8. Модифицировать `update_system()` для проверки локальных пакетов

Изменения в `update_system()`:

- Добавить проверку локального пакета перед проверкой сетевых репозиториев
- Если найден локальный пакет — использовать его
- Если не найден — использовать существующую логику

Этап 3: Интеграция с GUI ⏱️ 1-2 часа

Цель: Добавить элементы управления офлайн-обновлениями в GUI.

Файл: FSA-AstraInstall.py (модификация класса FSAAstraGUI)

Задачи:

1. Добавить кнопку "Подготовить офлайн-обновление" на вкладку "Обновление ОС"
2. Добавить индикатор статуса локальных пакетов
3. Добавить диалог выбора версии для подготовки пакета
4. Добавить прогресс-бар для процесса подготовки пакета
5. Добавить метод `_prepare_offline_update_package_gui()`
6. Добавить метод `_check_offline_packages_status()`
7. Обновить метод `create_update_process_subtab()`

Элементы GUI:

- Кнопка "Подготовить офлайн-обновление"
- Label с информацией о найденных локальных пакетах
- Checkbox "Использовать только локальный пакет"

Этап 4: Создание README для AstraPack/SystemUpdates/ ⏱️ 30 минут

Цель: Документировать использование офлайн-пакетов.

Файл: AstraPack/SystemUpdates/README.md (новый файл)

Содержание:

- Описание офлайн-пакетов
- Инструкция по подготовке пакета
- Инструкция по использованию пакета
- Требования и ограничения
- Примеры использования

Этап 5: Тестирование ⏱️ 2-3 часа

Цель: Проверить работоспособность системы офлайн-обновлений.

Тесты:

1. Подготовка офлайн-пакета для Astra Linux 1.7
2. Подготовка офлайн-пакета для Astra Linux 1.8
3. Установка обновлений из локального пакета (1.7)
4. Установка обновлений из локального пакета (1.8)
5. Проверка автоматического определения версии
6. Проверка fallback на интернет-обновление при отсутствии пакета
7. Проверка восстановления sources.list после обновления
8. Проверка полного офлайн-сценария (ОС + компоненты)

Этап 6: Документирование ⏱️ 1 час

Цель: Обновить документацию проекта.

Файлы:

- README.md — добавить раздел об офлайн-обновлениях
- HELPME.md — добавить инструкцию по использованию
- CHRONOLOGY.md — добавить запись о реализации

💻 Детальные изменения кода

Изменение 1: Класс OfflineUpdatePackager

Расположение: FSA-AstraInstall.py, после класса SystemUpdater (после строки ~37900)

```
#
==========================================================================
==
# ОФЛАЙН-ОБНОВЛЕНИЯ СИСТЕМЫ
#
==========================================================================
==

class OfflineUpdatePackager:
    """
    Класс для подготовки офлайн-пакетов обновлений системы

    Создает архивы с deb-файлами и локальным APT-репозиторием
    для обновления системы без доступа к интернету.
    """

    def __init__(self):
        """Инициализация OfflineUpdatePackager"""
        self.temp_dir = None
        self.package_list = []
        self.metadata = {}

    def prepare_offline_package(self, platform_version=None,
output_path=None):
        """
        Подготовка офлайн-пакета обновлений

        Args:
            platform_version: Версия платформы ('1-7', '1-8' или None для
автоопределения)
            output_path: Путь для сохранения пакета (по умолчанию
AstraPack/SystemUpdates/)

        Returns:
            bool: True если пакет успешно создан, False в случае ошибки
        """
        try:
            # 1. Определяем версию платформы
            if platform_version is None:
                platform_version = detect_astra_version()
                if platform_version is None:
```

```
                print("[ERROR] Не удалось определить версию системы")
                return False

        print(f"[INFO] Подготовка офлайн-пакета для Astra Linux
{platform_version}")

        # 2. Создаем временную директорию
        self.temp_dir = tempfile.mkdtemp(prefix='offline-update-prep-
')
        print(f"[INFO] Временная директория: {self.temp_dir}")

        # 3. Получаем список пакетов для обновления
        print("[INFO] Получение списка пакетов для обновления...")
        if not self._get_upgradable_packages():
            print("[ERROR] Не удалось получить список пакетов")
            return False

        if not self.package_list:
            print("[INFO] Нет доступных обновлений")
            return False

        print(f"[INFO] Найдено {len(self.package_list)} пакетов для
обновления")

        # 4. Скачиваем пакеты
        print("[INFO] Скачивание пакетов...")
        if not self._download_packages():
            print("[ERROR] Не удалось скачать пакеты")
            return False

        # 5. Создаем структуру репозитория
        print("[INFO] Создание структуры локального репозитория...")
        if not self._create_repository_structure():
            print("[ERROR] Не удалось создать структуру репозитория")
            return False

        # 6. Генерируем индексы репозитория
        print("[INFO] Генерация индексов репозитория...")
        if not self._generate_repository_indexes():
            print("[ERROR] Не удалось сгенерировать индексы")
            return False

        # 7. Создаем метаданные
        print("[INFO] Создание метаданных...")
        self._create_metadata(platform_version)

        # 8. Упаковываем в архив
        print("[INFO] Упаковка в архив...")
        if output_path is None:
            # Определяем путь к AstraPack/SystemUpdates/
            script_dir = os.path.dirname(os.path.abspath(__file__))
            astrapack_dir = os.path.join(script_dir,
ASTRAPACK_DIR_NAME)
            systemupdates_dir = os.path.join(astrapack_dir,
```

```
'SystemUpdates')

                # Создаем директорию если не существует
                os.makedirs(systemupdates_dir, exist_ok=True)

                # Формируем имя файла
                date_str = datetime.datetime.now().strftime("%Y%m%d")
                output_filename = f"offline-update-{platform_version}-
{date_str}.tar.gz"
                output_path = os.path.join(systemupdates_dir,
output_filename)

            if not self._create_archive(output_path):
                print("[ERROR] Не удалось создать архив")
                return False

            print(f"[OK] Офлайн-пакет успешно создан: {output_path}")
            return True

        except Exception as e:
            print(f"[ERROR] Ошибка подготовки офлайн-пакета: {e}")
            import traceback
            traceback.print_exc()
            return False

        finally:
            # Очищаем временные файлы
            if self.temp_dir and os.path.exists(self.temp_dir):
                try:
                    shutil.rmtree(self.temp_dir)
                    print("[INFO] Временные файлы очищены")
                except Exception as e:
                    print(f"[WARNING] Не удалось очистить временные файлы:
{e}")

    def _get_upgradable_packages(self):
        """
        Получение списка пакетов для обновления

        Returns:
            bool: True если список получен, False в случае ошибки
        """
        try:
            # Обновляем списки пакетов
            print("[INFO] Обновление списков пакетов...")
            result = subprocess.run(
                ['apt-get', 'update'],
                capture_output=True,
                text=True,
                timeout=300
            )

            if result.returncode != 0:
                print(f"[WARNING] apt-get update вернул код
```

```
{result.returncode}")
                print(f"[WARNING] {result.stderr}")

            # Получаем список обновляемых пакетов
            result = subprocess.run(
                ['apt', 'list', '--upgradable'],
                capture_output=True,
                text=True,
                timeout=60
            )

            if result.returncode != 0:
                print(f"[ERROR] Не удалось получить список обновлений")
                return False

            # Парсим вывод
            lines = result.stdout.strip().split('\n')
            self.package_list = []

            for line in lines[1:]:  # Пропускаем заголовок
                if not line.strip():
                    continue

                # Формат: package_name/suite version arch [upgradable
from: old_version]
                parts = line.split('/')
                if len(parts) >= 1:
                    package_name = parts[0].strip()
                    if package_name:
                        self.package_list.append(package_name)

            return True

        except subprocess.TimeoutExpired:
            print("[ERROR] Таймаут при получении списка пакетов")
            return False
        except Exception as e:
            print(f"[ERROR] Ошибка получения списка пакетов: {e}")
            return False

    def _download_packages(self):
        """
        Скачивание всех пакетов и их зависимостей

        Returns:
            bool: True если пакеты скачаны, False в случае ошибки
        """
        try:
            # Создаем директорию для deb-файлов
            deb_dir = os.path.join(self.temp_dir, 'deb', 'pool', 'main')
            os.makedirs(deb_dir, exist_ok=True)

            # Скачиваем все пакеты для обновления через apt-get download
            # Используем --download-only для dist-upgrade чтобы получить
```

```
все зависимости
            print("[INFO] Скачивание пакетов через apt-get...")

            # Создаем временную директорию для скачивания
            download_dir = tempfile.mkdtemp(prefix='apt-download-')

            try:
                # Выполняем dist-upgrade с флагом --download-only
                env = os.environ.copy()
                env['DEBIAN_FRONTEND'] = 'noninteractive'

                result = subprocess.run(
                    [
                        'apt-get', 'dist-upgrade',
                        '--download-only',
                        '-y',
                        '-o', f'Dir::Cache::Archives={download_dir}',
                        '-o', 'Dpkg::Options::=--force-confdef',
                        '-o', 'Dpkg::Options::=--force-confold'
                    ],
                    capture_output=True,
                    text=True,
                    timeout=3600,
                    env=env
                )

                if result.returncode != 0:
                    print(f"[WARNING] apt-get dist-upgrade --download-only
вернул код {result.returncode}")
                    print(f"[WARNING] {result.stderr}")
                    # Продолжаем, так как некоторые пакеты могли быть
скачаны

                # Копируем скачанные deb-файлы
                deb_files = [f for f in os.listdir(download_dir) if
f.endswith('.deb')]

                if not deb_files:
                    print("[ERROR] Не найдено скачанных deb-файлов")
                    return False

                print(f"[INFO] Скачано {len(deb_files)} deb-файлов")

                # Копируем файлы в структуру пакета
                for deb_file in deb_files:
                    src = os.path.join(download_dir, deb_file)
                    # Организуем по первой букве (как в Debian
репозиториях)
                    first_letter = deb_file[0].lower()
                    if first_letter.startswith('lib'):
                        first_letter = 'lib' + deb_file[3].lower()

                    dest_dir = os.path.join(deb_dir, first_letter)
                    os.makedirs(dest_dir, exist_ok=True)
```

```
                    dest = os.path.join(dest_dir, deb_file)
                    shutil.copy2(src, dest)

                print(f"[OK] Пакеты скопированы в {deb_dir}")
                return True

            finally:
                # Очищаем временную директорию скачивания
                if os.path.exists(download_dir):
                    shutil.rmtree(download_dir)

        except subprocess.TimeoutExpired:
            print("[ERROR] Таймаут при скачивании пакетов")
            return False
        except Exception as e:
            print(f"[ERROR] Ошибка скачивания пакетов: {e}")
            import traceback
            traceback.print_exc()
            return False

    def _create_repository_structure(self):
        """
        Создание структуры локального APT-репозитория

        Returns:
            bool: True если структура создана, False в случае ошибки
        """
        try:
            # Создаем структуру директорий
            repo_dir = os.path.join(self.temp_dir, 'repo', 'dists',
'local-update', 'main', 'binary-amd64')
            os.makedirs(repo_dir, exist_ok=True)

            print(f"[OK] Структура репозитория создана в {repo_dir}")
            return True

        except Exception as e:
            print(f"[ERROR] Ошибка создания структуры репозитория: {e}")
            return False

    def _generate_repository_indexes(self):
        """
        Генерация индексов репозитория (Packages, Packages.gz, Release)

        Returns:
            bool: True если индексы созданы, False в случае ошибки
        """
        try:
            deb_dir = os.path.join(self.temp_dir, 'deb', 'pool', 'main')
            repo_dir = os.path.join(self.temp_dir, 'repo', 'dists',
'local-update', 'main', 'binary-amd64')

            # Генерируем файл Packages с помощью dpkg-scanpackages
```

```
            print("[INFO] Генерация индекса Packages...")

            packages_file = os.path.join(repo_dir, 'Packages')

            with open(packages_file, 'w') as f:
                result = subprocess.run(
                    ['dpkg-scanpackages', '.', '/dev/null'],
                    cwd=deb_dir,
                    capture_output=True,
                    text=True,
                    timeout=300
                )

                if result.returncode != 0:
                    print(f"[ERROR] dpkg-scanpackages вернул код
{result.returncode}")
                    print(f"[ERROR] {result.stderr}")
                    return False

                f.write(result.stdout)

            print(f"[OK] Файл Packages создан: {packages_file}")

            # Сжимаем файл Packages
            print("[INFO] Создание Packages.gz...")
            packages_gz = os.path.join(repo_dir, 'Packages.gz')

            with open(packages_file, 'rb') as f_in:
                with gzip.open(packages_gz, 'wb') as f_out:
                    shutil.copyfileobj(f_in, f_out)

            print(f"[OK] Файл Packages.gz создан: {packages_gz}")

            # Создаем файл Release
            print("[INFO] Создание файла Release...")
            release_file = os.path.join(repo_dir, 'Release')

            with open(release_file, 'w') as f:
                f.write(f"Archive: local-update\n")
                f.write(f"Component: main\n")
                f.write(f"Architecture: amd64\n")
                f.write(f"Date: {datetime.datetime.now().strftime('%a, %d
%b %Y %H:%M:%S %z')}\n")

            print(f"[OK] Файл Release создан: {release_file}")

            return True

        except subprocess.TimeoutExpired:
            print("[ERROR] Таймаут при генерации индексов")
            return False
        except Exception as e:
            print(f"[ERROR] Ошибка генерации индексов: {e}")
            import traceback
```

```
            traceback.print_exc()
            return False

    def _create_metadata(self, platform_version):
        """
        Создание метаданных пакета

        Args:
            platform_version: Версия платформы ('1-7' или '1-8')
        """
        try:
            # Подсчитываем размер пакета
            deb_dir = os.path.join(self.temp_dir, 'deb')
            total_size = 0
            packages_count = 0

            for root, dirs, files in os.walk(deb_dir):
                for file in files:
                    if file.endswith('.deb'):
                        file_path = os.path.join(root, file)
                        total_size += os.path.getsize(file_path)
                        packages_count += 1

            total_size_mb = total_size / (1024 * 1024)

            # Определяем базовую систему и версию GLIBC
            if platform_version == '1-7':
                base_system = "Debian Buster"
                glibc_version = "2.28"
            elif platform_version == '1-8':
                base_system = "Debian Bookworm"
                glibc_version = "2.36"
            else:
                base_system = "Unknown"
                glibc_version = "Unknown"

            # Получаем версию проекта
            project_version = "Unknown"
            version_file = os.path.join(os.path.dirname(__file__),
'Version.txt')
            if os.path.exists(version_file):
                try:
                    with open(version_file, 'r') as f:
                        project_version = f.read().strip()
                except:
                    pass

            # Формируем метаданные
            self.metadata = {
                "version": platform_version,
                "date": datetime.datetime.now().strftime("%Y-%m-%d"),
                "packages_count": packages_count,
                "total_size_mb": round(total_size_mb, 2),
                "base_system": base_system,
```

```
                "glibc_version": glibc_version,
                "created_by": f"FSA-AstraInstall {project_version}"
            }

            # Сохраняем метаданные в файл
            metadata_file = os.path.join(self.temp_dir, 'metadata.json')
            with open(metadata_file, 'w', encoding='utf-8') as f:
                json.dump(self.metadata, f, indent=2, ensure_ascii=False)

            print(f"[OK] Метаданные созданы: {self.metadata}")

        except Exception as e:
            print(f"[WARNING] Ошибка создания метаданных: {e}")
            # Не критичная ошибка, продолжаем

    def _create_archive(self, output_path):
        """
        Упаковка в tar.gz архив

        Args:
            output_path: Путь для сохранения архива

        Returns:
            bool: True если архив создан, False в случае ошибки
        """
        try:
            print(f"[INFO] Создание архива: {output_path}")

            # Создаем README.txt
            readme_file = os.path.join(self.temp_dir, 'README.txt')
            with open(readme_file, 'w', encoding='utf-8') as f:
                f.write("=" * 70 + "\n")
                f.write("ОФЛАЙН-ПАКЕТ ОБНОВЛЕНИЙ FSA-AstraInstall\n")
                f.write("=" * 70 + "\n\n")
                f.write(f"Версия системы: Astra Linux
{self.metadata.get('version', 'Unknown')}\n")
                f.write(f"Дата создания: {self.metadata.get('date',
'Unknown')}\n")
                f.write(f"Количество пакетов:
{self.metadata.get('packages_count', 'Unknown')}\n")
                f.write(f"Размер: {self.metadata.get('total_size_mb',
'Unknown')} MB\n\n")
                f.write("ИНСТРУКЦИЯ ПО ИСПОЛЬЗОВАНИЮ:\n")
                f.write("-" * 70 + "\n\n")
                f.write("1. Скопируйте этот архив на целевую систему\n")
                f.write("2. Поместите в директорию
AstraPack/SystemUpdates/\n")
                f.write("3. Запустите FSA-AstraInstall\n")
                f.write("4. Нажмите 'Обновить систему'\n")
                f.write("5. Система автоматически обнаружит и использует
локальный пакет\n\n")
                f.write("ВНИМАНИЕ:\n")
                f.write("-" * 70 + "\n\n")
                f.write("- Пакет предназначен только для указанной версии
```

```
Astra Linux\n")
                f.write("- Не распаковывайте архив вручную\n")
                f.write("- Для обновления требуются права root\n\n")

            # Создаем tar.gz архив
            with tarfile.open(output_path, 'w:gz') as tar:
                # Добавляем все содержимое temp_dir
                for item in os.listdir(self.temp_dir):
                    item_path = os.path.join(self.temp_dir, item)
                    tar.add(item_path, arcname=item)

            # Проверяем размер архива
            archive_size = os.path.getsize(output_path)
            archive_size_mb = archive_size / (1024 * 1024)

            print(f"[OK] Архив создан: {output_path}")
            print(f"[OK] Размер архива: {archive_size_mb:.2f} MB")

            return True

        except Exception as e:
            print(f"[ERROR] Ошибка создания архива: {e}")
            import traceback
            traceback.print_exc()
            return False
```

Изменение 2: Модификация класса SystemUpdater

Расположение: FSA-AstraInstall.py, класс SystemUpdater

Добавить методы (после метода `_check_network_repositories()`):

```
def _find_offline_update_package(self):
    """
    Поиск офлайн-пакета обновлений для текущей версии системы

    Returns:
        str или None: Путь к пакету если найден, None если не найден
    """
    try:
        # Определяем версию системы
        platform_version = detect_astra_version()
        if platform_version is None:
            print("[INFO] Не удалось определить версию системы для поиска
офлайн-пакета")
            return None

        print(f"[INFO] Поиск офлайн-пакета для Astra Linux
{platform_version}...")

        # Определяем путь к AstraPack/SystemUpdates/
```

```
        script_dir = os.path.dirname(os.path.abspath(__file__))
        astrapack_dir = os.path.join(script_dir, ASTRAPACK_DIR_NAME)
        systemupdates_dir = os.path.join(astrapack_dir, 'SystemUpdates')

        if not os.path.exists(systemupdates_dir):
            print(f"[INFO] Директория {systemupdates_dir} не существует")
            return None

        # Ищем файлы вида offline-update-1-7-*.tar.gz или offline-update-
1-8-*.tar.gz
        pattern = f"offline-update-{platform_version}-*.tar.gz"
        matching_files = []

        for filename in os.listdir(systemupdates_dir):
            if fnmatch.fnmatch(filename, pattern):
                file_path = os.path.join(systemupdates_dir, filename)
                # Добавляем с датой модификации для сортировки
                mtime = os.path.getmtime(file_path)
                matching_files.append((mtime, file_path, filename))

        if not matching_files:
            print(f"[INFO] Офлайн-пакет для версии {platform_version} не
найден")
            return None

        # Сортируем по дате модификации (новые первые)
        matching_files.sort(reverse=True)

        # Берем самый новый
        newest_package = matching_files[0][1]
        newest_filename = matching_files[0][2]

        print(f"[OK] Найден офлайн-пакет: {newest_filename}")
        return newest_package

    except Exception as e:
        print(f"[ERROR] Ошибка поиска офлайн-пакета: {e}")
        return None

def _extract_offline_package(self, package_path, extract_to):
    """
    Распаковка офлайн-пакета

    Args:
        package_path: Путь к архиву
        extract_to: Директория для распаковки

    Returns:
        bool: True если распаковано успешно, False в случае ошибки
    """
    try:
        print(f"[INFO] Распаковка пакета: {package_path}")
        print(f"[INFO] Целевая директория: {extract_to}")
```

```
        # Создаем целевую директорию
        os.makedirs(extract_to, exist_ok=True)

        # Распаковываем архив
        with tarfile.open(package_path, 'r:gz') as tar:
            tar.extractall(extract_to)

        print(f"[OK] Пакет распакован в {extract_to}")

        # Проверяем метаданные
        metadata_file = os.path.join(extract_to, 'metadata.json')
        if os.path.exists(metadata_file):
            with open(metadata_file, 'r', encoding='utf-8') as f:
                metadata = json.load(f)
                print(f"[INFO] Пакет содержит
{metadata.get('packages_count', 'Unknown')} пакетов")
                print(f"[INFO] Размер: {metadata.get('total_size_mb',
'Unknown')} MB")
                print(f"[INFO] Дата создания: {metadata.get('date',
'Unknown')}")

        return True

    except Exception as e:
        print(f"[ERROR] Ошибка распаковки пакета: {e}")
        import traceback
        traceback.print_exc()
        return False

def _setup_local_repository(self, package_dir):
    """
    Настройка локального репозитория

    Args:
        package_dir: Директория с распакованным пакетом

    Returns:
        str или None: Путь к локальному репозиторию если успешно, None в
случае ошибки
    """
    try:
        # Копируем deb-файлы и структуру репозитория в /opt/local-apt-
repo/
        local_repo_path = '/opt/local-apt-repo'

        print(f"[INFO] Создание локального репозитория в
{local_repo_path}")

        # Удаляем старый репозиторий если существует
        if os.path.exists(local_repo_path):
            print(f"[INFO] Удаление старого репозитория...")
            shutil.rmtree(local_repo_path)

        # Создаем новый репозиторий
```

```
        os.makedirs(local_repo_path, exist_ok=True)

        # Копируем deb-файлы
        deb_source = os.path.join(package_dir, 'deb')
        deb_dest = os.path.join(local_repo_path, 'deb')

        if not os.path.exists(deb_source):
            print(f"[ERROR] Директория с deb-файлами не найдена:
{deb_source}")
            return None

        print(f"[INFO] Копирование deb-файлов...")
        shutil.copytree(deb_source, deb_dest)

        # Копируем структуру репозитория
        repo_source = os.path.join(package_dir, 'repo')
        repo_dest = os.path.join(local_repo_path, 'repo')

        if not os.path.exists(repo_source):
            print(f"[ERROR] Структура репозитория не найдена:
{repo_source}")
            return None

        print(f"[INFO] Копирование структуры репозитория...")
        shutil.copytree(repo_source, repo_dest)

        print(f"[OK] Локальный репозиторий создан в {local_repo_path}")
        return local_repo_path

    except Exception as e:
        print(f"[ERROR] Ошибка настройки локального репозитория: {e}")
        import traceback
        traceback.print_exc()
        return None

def _backup_sources_list(self):
    """
    Создание резервной копии sources.list

    Returns:
        bool: True если копия создана, False в случае ошибки
    """
    try:
        sources_list = '/etc/apt/sources.list'
        backup_file = '/etc/apt/sources.list.fsa-backup'

        if not os.path.exists(sources_list):
            print(f"[WARNING] Файл {sources_list} не существует")
            return True

        print(f"[INFO] Создание резервной копии {sources_list}")
        shutil.copy2(sources_list, backup_file)
        print(f"[OK] Резервная копия создана: {backup_file}")
```

```
        return True

    except Exception as e:
        print(f"[ERROR] Ошибка создания резервной копии: {e}")
        return False

def _add_local_repository_to_sources(self, local_repo_path):
    """
    Добавление локального репозитория в sources.list

    Args:
        local_repo_path: Путь к локальному репозиторию

    Returns:
        bool: True если добавлено успешно, False в случае ошибки
    """
    try:
        sources_list = '/etc/apt/sources.list'

        print(f"[INFO] Добавление локального репозитория в
{sources_list}")

        # Читаем текущий sources.list
        existing_content = ""
        if os.path.exists(sources_list):
            with open(sources_list, 'r') as f:
                existing_content = f.read()

        # Формируем запись для локального репозитория
        local_repo_entry = f"\n# FSA-AstraInstall: Локальный офлайн-
репозиторий\n"
        local_repo_entry += f"deb [trusted=yes]
file://{local_repo_path}/deb/pool/main ./\n"

        # Записываем обновленный sources.list
        with open(sources_list, 'w') as f:
            # Комментируем существующие репозитории
            for line in existing_content.split('\n'):
                if line.strip() and not line.strip().startswith('#'):
                    f.write(f"# {line}\n")
                else:
                    f.write(f"{line}\n")

            # Добавляем локальный репозиторий
            f.write(local_repo_entry)

        print(f"[OK] Локальный репозиторий добавлен в sources.list")
        return True

    except Exception as e:
        print(f"[ERROR] Ошибка добавления локального репозитория: {e}")
        import traceback
        traceback.print_exc()
        return False
```

```python
def _restore_sources_list(self):
    """
    Восстановление оригинального sources.list из резервной копии

    Returns:
        bool: True если восстановлено успешно, False в случае ошибки
    """
    try:
        sources_list = '/etc/apt/sources.list'
        backup_file = '/etc/apt/sources.list.fsa-backup'

        if not os.path.exists(backup_file):
            print(f"[WARNING] Резервная копия {backup_file} не найдена")
            return True

        print(f"[INFO] Восстановление {sources_list} из резервной копии")
        shutil.copy2(backup_file, sources_list)

        # Удаляем резервную копию
        os.remove(backup_file)

        print(f"[OK] Файл sources.list восстановлен")
        return True

    except Exception as e:
        print(f"[ERROR] Ошибка восстановления sources.list: {e}")
        return False

def _update_from_offline_package(self, package_path, dry_run=False):
    """
    Обновление системы из офлайн-пакета

    Args:
        package_path: Путь к архиву с пакетом
        dry_run: Режим тестирования (без реальных изменений)

    Returns:
        bool: True если обновление успешно, False в случае ошибки
    """
    temp_extract_dir = None
    local_repo_path = None

    try:
        print("\n" + "=" * 70)
        print("ОБНОВЛЕНИЕ СИСТЕМЫ ИЗ ОФЛАЙН-ПАКЕТА")
        print("=" * 70)

        if dry_run:
            print("[WARNING] РЕЖИМ ТЕСТИРОВАНИЯ: обновление НЕ
выполняется")
            print(f"[INFO] Будет использован пакет: {package_path}")
            return True
```

```
        # 1. Распаковываем пакет
        temp_extract_dir = tempfile.mkdtemp(prefix='offline-update-')
        if not self._extract_offline_package(package_path,
temp_extract_dir):
            return False

        # 2. Настраиваем локальный репозиторий
        local_repo_path = self._setup_local_repository(temp_extract_dir)
        if local_repo_path is None:
            return False

        # 3. Создаем резервную копию sources.list
        if not self._backup_sources_list():
            print("[WARNING] Не удалось создать резервную копию
sources.list, продолжаем...")

        # 4. Добавляем локальный репозиторий в sources.list
        if not self._add_local_repository_to_sources(local_repo_path):
            return False

        # 5. Обновляем списки пакетов
        print("\n[PROCESS] Обновление списков пакетов из локального
репозитория...")
        update_cmd = ['apt-get', 'update']
        result = self.run_command_with_interactive_handling(update_cmd,
False, gui_terminal=True)

        if result != 0:
            print("[ERROR] Ошибка обновления списков пакетов")
            return False

        # 6. Выполняем обновление системы
        print("\n[START] Обновление системы из локального репозитория...")
        upgrade_cmd = ['apt-get', 'dist-upgrade', '-y',
                       '-o', 'Dpkg::Options::=--force-confdef',
                       '-o', 'Dpkg::Options::=--force-confold',
                       '-o', 'Dpkg::Options::=--force-confmiss']
        result = self.run_command_with_interactive_handling(upgrade_cmd,
False, gui_terminal=True)

        if result != 0:
            print("[ERROR] Ошибка обновления системы")
            return False

        print("[OK] Система успешно обновлена из локального репозитория")

        # 7. Автоматическая очистка
        print("\n[CLEANUP] Автоматическая очистка ненужных пакетов...")
        autoremove_cmd = ['apt-get', 'autoremove', '-y',
                          '-o', 'Dpkg::Options::=--force-confdef',
                          '-o', 'Dpkg::Options::=--force-confold',
                          '-o', 'Dpkg::Options::=--force-confmiss']
        autoremove_result =
self.run_command_with_interactive_handling(autoremove_cmd, False,
```

```
gui_terminal=True)

        if autoremove_result == 0:
            print("[OK] Ненужные пакеты успешно удалены")
        else:
            print("[WARNING] Предупреждение: не удалось удалить ненужные
пакеты")

        return True

    except Exception as e:
        print(f"[ERROR] Ошибка обновления из офлайн-пакета: {e}")
        import traceback
        traceback.print_exc()
        return False

    finally:
        # Восстанавливаем sources.list
        print("\n[CLEANUP] Восстановление оригинального sources.list...")
        self._restore_sources_list()

        # Обновляем списки пакетов из оригинальных репозиториев
        print("[CLEANUP] Обновление списков пакетов из оригинальных
репозиториев...")
        try:
            subprocess.run(['apt-get', 'update'], capture_output=True,
timeout=60)
        except:
            pass

        # Очищаем временные файлы
        if temp_extract_dir and os.path.exists(temp_extract_dir):
            try:
                shutil.rmtree(temp_extract_dir)
                print("[OK] Временные файлы очищены")
            except Exception as e:
                print(f"[WARNING] Не удалось очистить временные файлы:
{e}")

        # Очищаем локальный репозиторий
        if local_repo_path and os.path.exists(local_repo_path):
            try:
                shutil.rmtree(local_repo_path)
                print("[OK] Локальный репозиторий удален")
            except Exception as e:
                print(f"[WARNING] Не удалось удалить локальный
репозиторий: {e}")
```

Модифицировать метод `update_system()` (строка ~37060):

Заменить начало метода:

```
def update_system(self, dry_run=False):
    """Обновление системы"""
    print("[PACKAGE] Обновление системы...")

    # НОВОЕ: Проверка наличия офлайн-пакета ПЕРЕД проверкой сетевых
репозиториев
    print("\n[CHECK] Шаг 1/4: Проверка наличия офлайн-пакета
обновлений...")
    offline_package = self._find_offline_update_package()

    if offline_package:
        print("[INFO] Обнаружен офлайн-пакет обновлений")
        print("[INFO] Будет использовано обновление из локального пакета")
        return self._update_from_offline_package(offline_package, dry_run)

    print("[INFO] Офлайн-пакет не найден, используем обновление через
интернет")

    # КРИТИЧНО: Проверка сетевых репозиториев перед обновлением
    print("\n[CHECK] Шаг 2/4: Проверка наличия сетевых репозиториев...")
    if not self._check_network_repositories():
        print("[ERROR] Нет сетевых репозиториев!")
        print("[ERROR] Обновление системы невозможно без сетевых
репозиториев")
        print("[INFO] Пожалуйста, настройте сетевые репозитории и
повторите попытку")
        print("[INFO] ИЛИ поместите офлайн-пакет в
AstraPack/SystemUpdates/")
        return False

    # КРИТИЧНО: Проверка наличия обновлений перед обновлением
    print("\n[CHECK] Шаг 3/4: Проверка наличия доступных обновлений...")
    # ... остальной код метода без изменений
```

📚 Примеры использования

Сценарий 1: Подготовка пакета на сервере с интернетом

```
# Вариант 1: Через GUI
sudo ./FSA-AstraInstall-1-7
# В GUI: Вкладка "Обновление ОС" → "Подготовить офлайн-обновление"
# Выбрать версию: "Автоопределение" или "Astra Linux 1.7"
# Нажать "Начать"
# Ждать завершения (может занять 10-30 минут)

# Вариант 2: Через консоль (будущая реализация)
sudo ./FSA-AstraInstall-1-7 --prepare-offline-update --version=1-7
```

Результат:

```
AstraPack/SystemUpdates/offline-update-1-7-20251230.tar.gz (2.1 GB)
```

Сценарий 2: Перенос на изолированную систему

```
# На сервере с интернетом (или на macOS)
cd /Volumes/FSA-PRJ/Project/FSA-AstraInstall
cp -r AstraPack/ /mnt/usb/

# ИЛИ по сети:
scp -r AstraPack/ user@isolated-system:/tmp/

# На изолированной системе
cd /tmp
# AstraPack/ уже содержит офлайн-пакет обновлений
```

Сценарий 3: Обновление системы без интернета

```
# На изолированной системе
sudo ./FSA-AstraInstall-1-7

# В GUI:
# 1. Вкладка "Обновление ОС" → "Проверить локальные пакеты"
#    Статус: "Найден пакет для 1-7: offline-update-1-7-20251230.tar.gz
(2.1 GB)"
#
# 2. Нажать "Обновить систему"
#    Система автоматически обнаружит локальный пакет и использует его
#
# Лог:
# [INFO] Обнаружен офлайн-пакет обновлений
# [INFO] Будет использовано обновление из локального пакета
# [INFO] Распаковка пакета...
# [INFO] Создание локального репозитория...
# [INFO] Обновление системы из локального репозитория...
# [OK] Система успешно обновлена из локального репозитория
```

Сценарий 4: Полная автономная установка (ОС + компоненты)

```
# На изолированной системе (полностью без интернета)
```

```
# 1. Запускаем FSA-AstraInstall
sudo ./FSA-AstraInstall-1-7

# 2. Обновляем систему (использует локальный пакет)
#    Вкладка "Обновление ОС" → "Обновить систему"

# 3. Устанавливаем компоненты (из AstraPack/)
#    Вкладка "Установка программ" → выбираем компоненты → "Установить"

# Результат: полностью готовая сборка без интернета!
```

🧪 Тестирование

Тест 1: Подготовка пакета для Astra Linux 1.7

Цель: Проверить создание офлайн-пакета для версии 1.7

Шаги:

1. На системе Astra Linux 1.7 с интернетом запустить FSA-AstraInstall
2. Перейти на вкладку "Обновление ОС"
3. Нажать "Подготовить офлайн-обновление"
4. Выбрать "Astra Linux 1.7"
5. Нажать "Начать"

Ожидаемый результат:

- Пакет создан в `AstraPack/SystemUpdates/offline-update-1-7-YYYYMMDD.tar.gz`
- Размер пакета 1.5-3 GB
- Содержит все необходимые deb-файлы
- Содержит структуру репозитория
- Содержит metadata.json с информацией о пакете

Тест 2: Подготовка пакета для Astra Linux 1.8

Цель: Проверить создание офлайн-пакета для версии 1.8

Шаги: Аналогично тесту 1, но выбрать "Astra Linux 1.8"

Ожидаемый результат: Пакет создан для версии 1.8

Тест 3: Обновление из локального пакета (1.7)

Цель: Проверить обновление системы из офлайн-пакета

Шаги:

1. На системе Astra Linux 1.7 без интернета поместить пакет в `AstraPack/SystemUpdates/`
2. Запустить FSA-AstraInstall

3. Нажать "Проверить локальные пакеты" (должен отобразиться статус "Найден пакет...")
4. Нажать "Обновить систему"

Ожидаемый результат:

- Система автоматически обнаруживает локальный пакет
- Выполняется обновление из локального репозитория
- sources.list восстанавливается после обновления
- Временные файлы очищены

Тест 4: Автоопределение версии

Цель: Проверить автоматический выбор нужного пакета

Шаги:

1. Поместить оба пакета (1-7 и 1-8) в `AstraPack/SystemUpdates/`
2. Запустить FSA-AstraInstall на системе 1.7
3. Нажать "Обновить систему"

Ожидаемый результат:

- Система автоматически выбирает пакет для версии 1.7
- Обновление выполняется из правильного пакета

Тест 5: Fallback на интернет-обновление

Цель: Проверить переключение на интернет при отсутствии локального пакета

Шаги:

1. На системе с интернетом удалить папку `AstraPack/SystemUpdates/`
2. Запустить FSA-AstraInstall
3. Нажать "Обновить систему"

Ожидаемый результат:

- Система не находит локальный пакет
- Автоматически переключается на обновление через интернет
- Обновление выполняется из сетевых репозиториев

Тест 6: Полный офлайн-сценарий

Цель: Проверить полную офлайн-установку (ОС + компоненты)

Шаги:

1. Подготовить офлайн-пакет на системе с интернетом
2. Скопировать весь `AstraPack/` на USB
3. На изолированной системе без интернета:

    - Обновить систему (из локального пакета)
    - Установить Wine (из AstraPack/Wine/)
    - Установить Astra.IDE (из AstraPack/Astra/)

Ожидаемый результат:

- Все операции выполнены успешно без интернета
- Система обновлена
- Компоненты установлены

Тест 7: Проверка восстановления sources.list

Цель: Убедиться, что sources.list корректно восстанавливается

Шаги:

1. Сохранить оригинальный `sources.list`
2. Выполнить обновление из локального пакета
3. Сравнить восстановленный `sources.list` с оригиналом

Ожидаемый результат:

- Файл восстановлен в исходное состояние
- Локальный репозиторий удален
- Оригинальные репозитории раскомментированы

⚠ Риски и митигация

Риск 1: Несовместимость версий

Описание: Использование пакета для неправильной версии (1.7 вместо 1.8)

Вероятность: Средняя
Влияние: Критическое (система может быть повреждена)

Митигация:

- Автоматическое определение версии через `detect_astra_version()`
- Проверка метаданных пакета перед использованием
- Явная индикация версии в имени файла
- Предупреждение в GUI при несовпадении версий

Риск 2: Недостаточно места на диске

Описание: Не хватает места для распаковки и установки обновлений

Вероятность: Средняя
Влияние: Высокое (обновление не выполнится)

Митигация:

- Проверка свободного места перед распаковкой (минимум 5 GB)
- Использование `/tmp` с очисткой после установки
- Информирование пользователя о требуемом пространстве
- Встроенная проверка в `SystemUpdater._check_disk_space()`

Риск 3: Повреждение sources.list

Описание: Файл sources.list не восстановился после обновления

Вероятность: Низкая
Влияние: Высокое (система теряет доступ к репозиториям)

Митигация:

- Создание резервной копии перед изменением
- Использование `try-finally` для гарантированного восстановления
- Проверка существования резервной копии
- Логирование всех операций с sources.list

Риск 4: Неполный набор пакетов

Описание: В пакете отсутствуют некоторые зависимости

Вероятность: Средняя
Влияние: Среднее (не все обновления установятся)

Митигация:

- Использование `apt-get dist-upgrade --download-only` (скачивает все зависимости)
- Проверка количества скачанных пакетов
- Логирование отсутствующих пакетов
- Возможность повторной подготовки пакета

Риск 5: Большой размер пакета

Описание: Пакет слишком большой для переноса (3-5 GB)

Вероятность: Высокая
Влияние: Низкое (неудобство)

Митигация:

- Использование tar.gz сжатия (уменьшает размер на 30-40%)
- Информирование пользователя о размере перед созданием
- Возможность создания только критичных обновлений (будущая опция)
- Поддержка разделения на части (будущая опция)

Риск 6: Долгое время подготовки

Описание: Подготовка пакета занимает 20-40 минут

Вероятность: Высокая
Влияние: Низкое (неудобство)

Митигация:

- Выполнение в отдельном потоке (GUI не блокируется)
- Прогресс-бар с индикацией этапов
- Информирование о примерном времени
- Возможность отмены процесса

📊 Оценка трудозатрат

Этап	Задача	Часы	Приоритет
1	Класс OfflineUpdatePackager	2-3	Высокий
2	Модификация SystemUpdater	2-3	Высокий
3	Интеграция с GUI	1-2	Средний
4	README для SystemUpdates	0.5	Низкий
5	Тестирование	2-3	Высокий
6	Документирование	1	Средний
ИТОГО		**8-12**	

📝 Чеклист реализации

Этап 1: OfflineUpdatePackager

- [ ] Создать класс OfflineUpdatePackager
- [ ] Реализовать prepare_offline_package()
- [ ] Реализовать _get_upgradable_packages()
- [ ] Реализовать _download_packages()
- [ ] Реализовать _create_repository_structure()
- [ ] Реализовать _generate_repository_indexes()
- [ ] Реализовать _create_metadata()
- [ ] Реализовать _create_archive()
- [ ] Добавить обработку ошибок
- [ ] Добавить логирование

Этап 2: Модификация SystemUpdater

- [ ] Добавить _find_offline_update_package()
- [ ] Добавить _extract_offline_package()
- [ ] Добавить _setup_local_repository()
- [ ] Добавить _backup_sources_list()

- ☐ Добавить _add_local_repository_to_sources()
- ☐ Добавить _restore_sources_list()
- ☐ Добавить _update_from_offline_package()
- ☐ Модифицировать update_system()
- ☐ Добавить обработку ошибок
- ☐ Добавить логирование

Этап 3: GUI интеграция

- ☐ Добавить кнопку "Подготовить офлайн-обновление"
- ☐ Добавить кнопку "Проверить локальные пакеты"
- ☐ Добавить Label статуса
- ☐ Реализовать _prepare_offline_update_package_gui()
- ☐ Реализовать _run_offline_package_preparation()
- ☐ Реализовать _on_offline_package_prepared()
- ☐ Реализовать _check_offline_packages_status()
- ☐ Добавить диалог выбора версии

Этап 4: Документация

- ☐ Создать AstraPack/SystemUpdates/README.md
- ☐ Обновить README.md проекта
- ☐ Обновить HELPME.md
- ☐ Обновить CHRONOLOGY.md
- ☐ Добавить примеры использования

Этап 5: Тестирование

- ☐ Тест: Подготовка пакета для 1.7
- ☐ Тест: Подготовка пакета для 1.8
- ☐ Тест: Обновление из локального пакета (1.7)
- ☐ Тест: Обновление из локального пакета (1.8)
- ☐ Тест: Автоопределение версии
- ☐ Тест: Fallback на интернет
- ☐ Тест: Полный офлайн-сценарий
- ☐ Тест: Восстановление sources.list

🎯 Критерии готовности (Definition of Done)

1. ✅ Класс OfflineUpdatePackager реализован и протестирован
2. ✅ SystemUpdater поддерживает локальные пакеты
3. ✅ GUI интеграция завершена
4. ✅ Автоматическое определение версии работает
5. ✅ Резервное копирование и восстановление sources.list работает
6. ✅ Все тесты пройдены успешно
7. ✅ Документация обновлена
8. ✅ README.md для SystemUpdates создан

9. ✅ Полный офлайн-сценарий работает
10. ✅ Код соответствует стилю проекта

📚 Дополнительные ресурсы

Документация Debian/APT:

- https://wiki.debian.org/DebianRepository/Setup
- https://manpages.debian.org/buster/dpkg-dev/dpkg-scanpackages.1.en.html
- https://www.debian.org/doc/manuals/repository-howto/repository-howto

Существующие решения:

- apt-offline: https://github.com/rickysarraf/apt-offline
- apt-mirror: https://apt-mirror.github.io/

Структура Debian репозитория:

```
dists/
  └── local-update/
      └── main/
          └── binary-amd64/
              ├── Packages
              ├── Packages.gz
              └── Release

pool/
  └── main/
      ├── a/
      │   └── apt_1.8.2.3_amd64.deb
      ├── b/
      │   └── bash_5.0-4_amd64.deb
      └── ...
```

Дата создания документа: 2025.12.30
Автор: AI Assistant (@LLM)
Статус: Готов к реализации